Arrays

Arrays

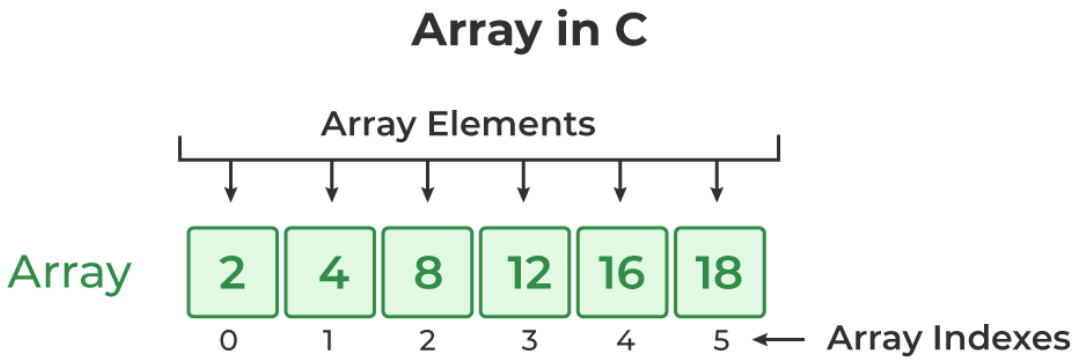
Definition: -

An array in C is a ***fixed-size collection of similar data items stored in contiguous memory locations.*** It can be used to store the collection of primitive data types such as int, char, float, etc., and also derived and user-defined data types such as pointers, structures, etc.

- One of the most used data structures in C programming
- Arrays are used to store **multiple values** in a **single variable name**, instead of declaring separate variables for each value.
- To create an array, define the data type (like int) and specify the name of the array followed by **square brackets []**. Eg- *int ar[];*
- To insert values to it, use a comma-separated list inside curly braces, and make sure all values are of the same data type: Eg - *int ar[] = {25, 50, 75, 100};*

Index numbers

- Each array element is referred to by its **index number**.
- Array indexes start with **0**: [0] is the first element. [1] is the second element, etc.
- For an array, int ar[]={2,4,8,12,16,18]



Declaration and Memory representation

C Array Declaration

- Have to declare the array like any other variable before using it.
- We can declare an array by specifying its name, the type of its elements, and the size of its dimensions.
- When we declare an array in C, the compiler allocates the memory block of the specified size to the array name.

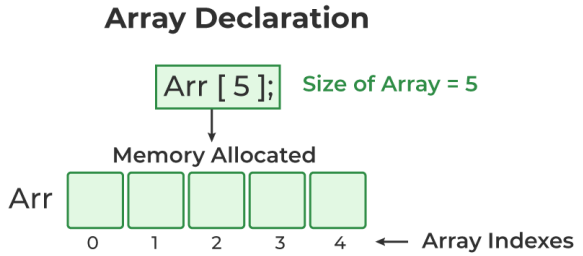
Syntax:

***data_type array_name* [size];**

or

***data_type array_name* [*size1*] [*size2*]...[*sizeN*];** where N is the number of dimensions [for ID, only one size like int ar[size1], for 2D, int ar [size1][size2]]

Memory representation



Array Declaration

Eg for array declaration

```
/*C Program to illustrate the array declaration */
#include <stdio.h>
int main()
 {
      int arr_int[5]; /*declaring array of integers*/
      char arr_char[5]; /* declaring array of characters */
      return 0;
 }
```

Array initialization

- Initialization is the process to assign some initial value to the variable.
- When the array is declared or allocated memory, the elements of the array contain some garbage value.
- So, we need to initialize the array to some meaningful value.
- There are multiple ways in which we can initialize an array in C.

1. Array Initialization with Declaration

- Initialize the array along with its declaration.
- Use an initializer list to initialize multiple elements of the array.
- An initializer list is the list of values enclosed within braces **{ }** separated b a comma.

Syntax:

***data_type array_name* [*size*] = {value1, value2, ... valueN};**

Eg - int arr[4] = {25, 50, 75, 100};

Array initialization contd…

2. Array Initialization with Declaration without Size

- If we initialize an array using an initializer list, we can skip declaring the size of the array as the compiler can automatically deduce the size of the array in these cases.
- The **size of the array** in these cases is **equal to the number of elements present in the initializer list** as the compiler can automatically deduce the size of the array.

Syntax:

data_type array_name[] = {elements separated by comma};

Eg-

```
int ar[]={1,2,3,4,5};
```

The size of the above arrays is 5 which is automatically deduced by the compiler.

Array initialization contd…

3. Array Initialization after Declaration (Using Loops)

- We initialize the array after the declaration by assigning the initial value to each element individually.
- We can use for loop, while loop, or do-while loop to assign the value to each element of the array.

```
for (int i = 0; i < N; i++)
 {
     array_name[i] = valuei;
 }
```

Adv and Disadv of arrays

Advantage of C Array

- **1) Code Optimization**: Less code to the access the data.
- **2) Ease of traversing**: By using the for loop, we can retrieve the elements of an array easily.
- **3) Ease of sorting**: To sort the elements of the array, we need a few lines of code only.
- **4) Random Access**: We can access any element randomly using the array.

Disadvantage of C Array

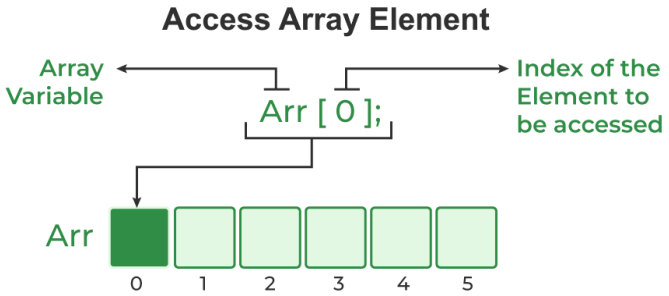
- **1) Fixed Size**: Whatever size, we define at the time of declaration of the array, we can't exceed the limit. So, it doesn't grow the size dynamically like LinkedList which we will learn later.

Accessing array elements

- We can access any element of an array in C using the **array subscript operator [ ] and the index value *i* of the element.**

  like *array_name* [*index*];

Important note - The indexing in the array always starts with 0, i.e., the **first element** is at index **0** and the **last element** is at **N – 1** where **N** is the number of elements in the array.



Accessing array elements

```
/ *C Program to illustrate element access using array subscript*/
 #include <stdio.h>
int main()
{
  int arr[5] = { 15, 25, 35, 45, 55 }; /* array declaration and initialization */
  printf("Element at arr[2]: %d\n", arr[2]); /*accessing element at index 2 i.e 3rd element */
  printf("Element at arr[4]: %d\n", arr[4]); /*accessing element at index 4 i.e last element */
  printf("Element at arr[0]: %d", arr[0]); /*accessing element at index 0 i.e first element */
  return 0;
}
```

Updating array elements

Update Array Element

- We can update the value of an element at the given index i in a similar way to accessing an element by using the array subscript operator **[ ]** and assignment operator **=**.

Syntax:

array_name[*i*] = *new_value*;

int arr[]={1,2,3,4,5}

arr[2]=10;

Accessing array elements

```
/ *C Program to illustrate element access using array subscript and loops*/
#include<stdio.h>
int main()
{
   int i=0;
   int marks[5]={20,30,40,50,60};      /*declaration and initialization of array */

   for(i=0;i<5;i++)                    /*traversal of array */
   {
    printf("%d \n",marks[i]);
    }
return 0;
}
```

Reading and printing array elements

```
void main ()
{
    int arr[20],i,n,;
    printf("Enter the size of the array?");
    scanf("%d",&n);
    printf("Enter the elements of the array?");
    for(i = 0; i<n; i++)
    {
        scanf("%d",&arr[i]);
    }
    printf(“\nThe elements of the array:-\n”);
    for(i=0; i<n; i++)
    {
        printf(“%d\t”, arr[i]);
    }
}
```

Printing the reverse of array elements

```
void main ()
{
    int arr[20],i,n,;
    printf("Enter the size of the array?");
    scanf("%d",&n);
    printf("Enter the elements of the array?
");
    for(i = 0; i<n; i++)
    {
        scanf("%d",&arr[i]);
    }
    printf(“\nThe elements of the array:-
\n”);
    for(i=0; i<n; i++)/* modify this line*/
    {
        printf(“%d\t”, arr[i]);
    }
}
```

Sorting the array elements

```
#include<stdio.h>
void main ()
{
   int i, j,temp;
   int a[10] = { 10, 9, 7, 101, 23, 44, 12, 78, 34, 23};
   for(i = 0; i<10; i++)
   {
      for(j = i+1; j<10; j++)
      {
         if(a[j] > a[i])
         {
            temp = a[i];
            a[i] = a[j];
            a[j] = temp;
         }
      }
   }
   printf("Printing Sorted Element List ...\n");

   for(i = 0; i<10; i++)
   {
      printf("%d\n",a[i]);
   }
}
```

Array of Characters

Array of Characters (Strings)

- In C, we store the words, i.e., a sequence of characters in the form of an array of characters terminated by a **NULL** character. These are called strings in C language.

Eg:

```
/* C Program to illustrate character arrays (strings)*/
#include <stdio.h>
int main()
{
    char arr[6] = { ‘C', ‘o', ‘m', ‘p', ‘u', ‘t’, ’e’, ’r’,'\0' }; //* creating array of character  of size 9*/
     int i = 0;
     while (arr[i])          /*printing string */
     {
           printf("%c", arr[i++]);
      }
     return 0;
}
```

Problems

1. Write a C program to print the binary equivalent of a given decimal number.
2. Write a C program to find the sum of elements of an array.
3. Write a C program to sort the array elements in descending order
4. Write a C program to find the largest and smallest elements in a array
5. Write a program in C to separate odd and even integers into separate arrays.
6. Write a C program to delete an element at a desired position from an array

